

Student name: Sarah Mello

Student No.: ST10534482

Module Name: Programming for Data Analytics 2 PDAN8412

POE: Part 1

Date: 14 September 2026

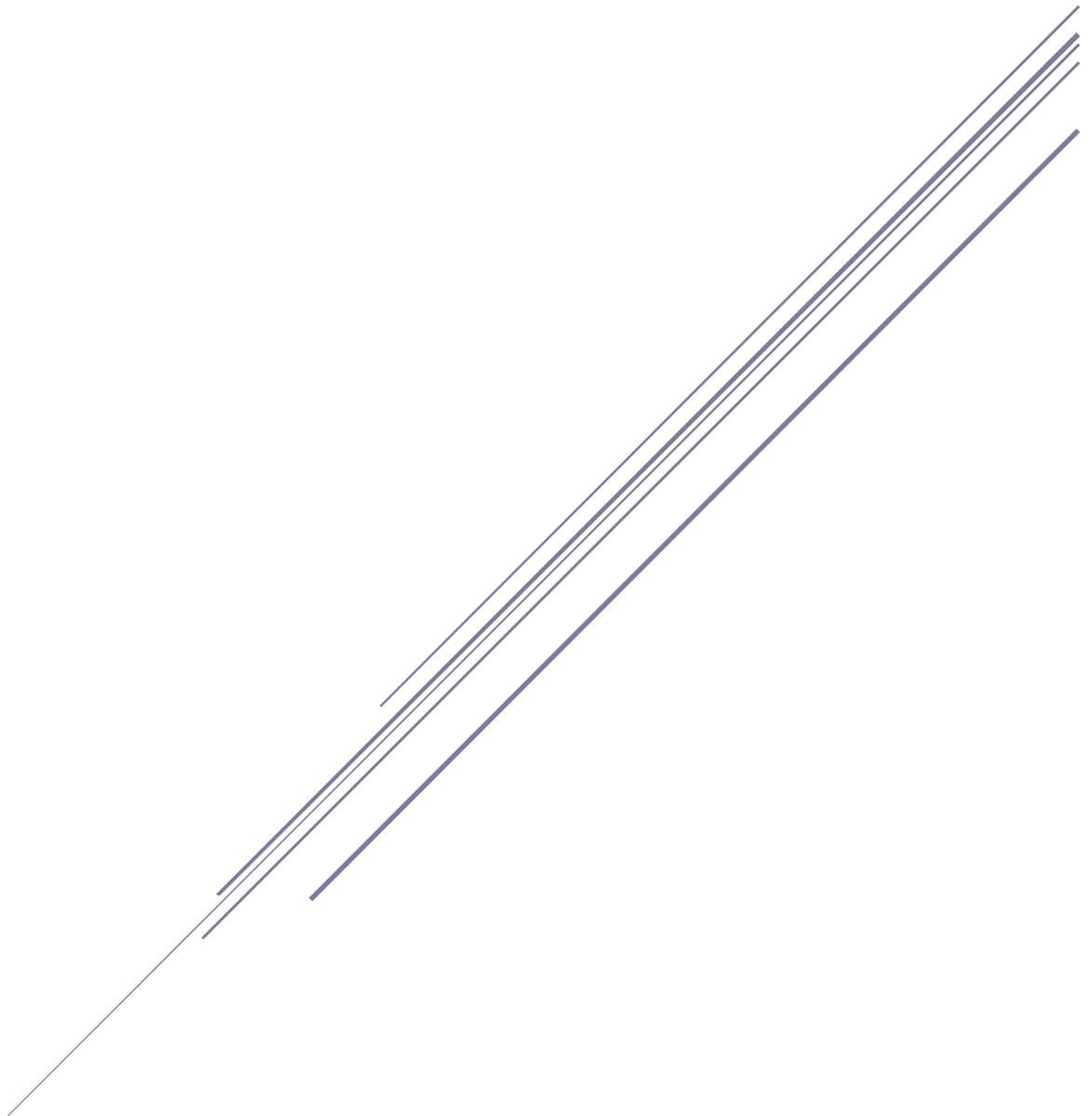


Table of contents

1. Project Context and objectives	2
2. Evaluation and Justification of using Spooky Authors identification	2
3. Analysis planning	3
4. Exploratory Data Analysis (EDA).....	5
5. Model training and analysis.....	7
6. Model evaluation and interpretation	8
Reference	11

Github Link: <https://github.com/ST10534482/S-Mello>

1. Project Context and objectives

ABC bookstore wanted to offer their customers a bot that they can interact with for fun. The primary function of the Bot called AuthorshipBot, is to analyse a given text excerpt and guess the author based on their unique writing style. To capture these stylistic nuances, the project recurrent Recurrent Neural Networks to build and train the model.

2. Evaluation and Justification of using Spooky Authors identification

- a. Reasons for the data to fit Recurrent Neural Network (RNN) model.
 - Order of the words: Kumar (2024) note that recurrent neural networks they are “designed to handle sequential data by retaining information from previous inputs in a hidden state”. While Long Short-Term Memory (LSTM) are designed to retain information for long sequences. Therefore, this will assist the bot to identify patterns of authors based on passive voice or the habits in punctuation.
 - Flexibility of the lengths of sentences: The lengths of sentences that humans write is different, therefore RNN can handle the different sentences.
- b. Quality of data
 - The dataset exceeds the minimum requirement of 10000, there are 19579 records in the dataset used.
 - The headings are labelled clearly and each block contains an author.
 - There are no missing data.
- c. Data traps avoidance
 - **Class Imbalance Trap** - Should one of the authors have more data than the others the model will be biased. The dataset is balanced with 7,900 for EAP, 5,635 for HPL and 6,044 for MWS. Therefore the data will be split equally and F1-Scores will be used instead of simple accuracy to make sure that the grading is fair.
 - **Noise Trap** – where there are capital letter and punctuation vocabulary, they will be lowercased and user regex filter in order to remove noise that can be caused by non-alphabets.
 - **New Word (OOV) Trap**: to avoid a crash by test data an out-of-vocabulary <OOV> token will be used to group new words safely.
 - **Length Trap**: sentences are not the same size, some are too long while some are too short therefore they will be capped at 150 words. Sentences that are too short, padding zeros will be added to make the 150 words.

Amrouni, S., Espenilla, L., Guincestre, A., and Mizrahi, P., 2017. Spooky-authors-identification [Computer software]. GitHub. [online] Available at: <<https://github.com/selimamrouni/spooky-authors-identification>> [Accessed 03 September 2026].

3. Analysis planning

a. Objective of the task

To create a bot that will guess an author based on writing style of the author that FANs can interact with for fun. An Long Short-Term Memory (LSTM) will be used and implemented with PySpark for data pipelines and for model execution TensorFlow will be used. Below is the flow of each step of the planning:

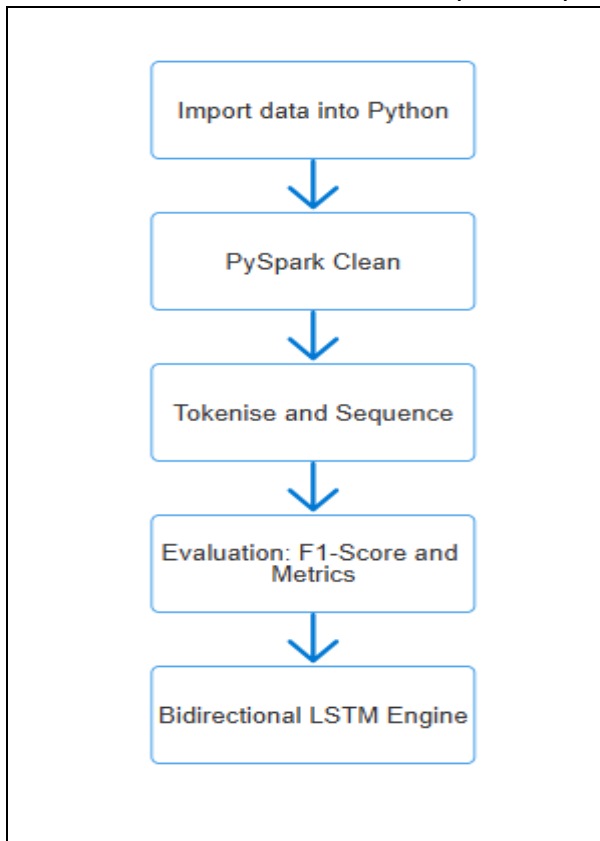


Figure 1: Analysis Planning

b. Exploratory Data Analysis (EDA) Plan

The following will be performed for EDA:

- Stylometric Feature Check - Calculation of punctuations such as exclamation marks, semicolons, etc.
- Class Distribution - Calculation of precise ratio of records per author. Stratified Shuffle Split (80/20) will be enforced so that validation sets mimic the training set distribution in a perfect manner.

- Sequence Length Quantile Analysis - Calculation of the 50th, 90th, and 99th percentiles of word count in every sentence. This is done so that more than 95% of the length of sentences are captured.

c. Data Cleaning & Feature Engineering

- Text Standardisation: A two-tier cleanup will be performed in order to keep the unique author's voices and is as follows:
 - Words will be standardised so that the writing styles of the authors is not altered. Therefore, for example words like "didn't" will be changed to "did not".
 - Some symbols that authors use will be preserved, and written in words. For example, this symbol ";" will be replaced with [semicolon] in order for the LSTM to track explicit habits of authors' punctuation.
- Vectorisation, Tokenisation, and Handling Traps
 - Hard-capping of vocabulary – Zipf's law states that some words are used all the time and a lot of words are used rarely most of the time (Geeksforgeeks, 2025). Based on Zipf's law of word frequency, vocabulary will be limited to top 20 000 unique words. Reason for this limitation is to prevent the model's parameter footprint to explode.
 - Out-of-Vocabulary mitigation – since there is a limitation of top unique 20 000 words, all the words outside that will be mapped to be explicit <OOV> token. This is done so that the bot does not throw errors when it comes across words that are rare.
- - Dynamic sequence padding – sentences will be capped at 150 words and sentences that have less than 150 words will be filled with neutral zeros.

d. Architecture Plan for deep learning model

- Embedding layer – words will be converted into numbers and placed into 128-dimensional space. Words that have similar meaning will be placed close together.
- Spatial Dropout 1D - Randomly turns off entire words (40% rate) during training. This prevents the model from memorizing specific words and therefore, looks closely to the context of the sentence.
- Bidirectional wrapper – sentences will be read backwards and forwards at the same time to assist the model to catch styling cues from the beginning and the end of sentences.

e. Training and optimisation strategy

- Loss function: According to Geeksforgeeks (2025) Sparse Categorical Crossentropy is referred to "a loss function commonly used in multi-class

classification problems in machine learning and deep learning and is particularly used when dealing with a large number of categories” therefore it will be used.

- Smart learning speed – Adam optimizer it is an optimizer that is used in deep learning (Geeksforgeeks, 2025). It starts at a speed (0.001) and slows down to avoid overshooting of the model's best settings.
- Auto-Stop - Training is capped at 15 rounds

f. Model evaluation plan

- Weighted F1-Score - It balances precision and recall for each author. This will assist with accuracy checks.
- Macro-Averaged Evaluation – to ensure that the bot makes good guesses of each author.
- Confusion Matrix – it will assist to indicate certain errors so that the model can be adjusted.

4. Exploratory Data Analysis (EDA)

a. Sequence Length Distribution Analysis

- Identified linguistic patterns – word count distribution was analysed across all 19579 records. The sentence length profiles vary significantly across different classic literature pieces: the median sentence contains only 25 words, but the maximum text fragment spikes to over 800 words.

```

--- 2. EDA: Class Distribution Strategy ---
+-----+-----+
|          author|count|
+-----+-----+
| I'm all soul and...| 1|
| and the supposit...| 1|
| "" who preached a...| 1|
| at this period o...| 1|
| ""It gave me the...| 1|
| that these Blasph...| 1|
|      Madame Lalande| 1|
| and I cannot con...| 1|
| one of the ""Eng...| 1|
| you have straigh...| 1|
| and we continued...| 1|
| and in a few bri...| 1|
|      and very happy| 1|
| turning abruptly...| 1|
| who art called o...| 1|
| who gave me this...| 1|
|      Mr. Wyatt.""| 1|
|      Woodville| 1|
| and returned wit...| 1|
| thet Afriky book?"| 1|
+-----+-----+
only showing top 20 rows

```

Figure 2: Class Distribution

```

--- 2. EDA: Sequence Length Quantile Analysis ---
+-----+-----+
|summary| token_count|
+-----+-----+
| mean| 25.29649113846468|
| 50%| 22|
| 90%| 46|
| 95%| 56|
| max| 674|
+-----+-----+

```

Figure 3: Sequence Length Quantile Analysis

- Model training direct connection – a 95th percentile threshold was calculated which was set at 136 words.
- Impact of the model execution – Statistics were mapped directly to Keras configuration by using MAX_SEQUENCE_LENGTH = 136. This mapping ensure that 95% of sentences are processed completely intact without any truncation. Since the maximum of sentences is 136 words, those with less than the maximum were filled with zeros (padding='post').

b. Identification and removal of outliers

- Identifying linguistic pattern:
 - i. Shorthand Noise Outliers - Fragments consisting of only 1 or 2 words for example, standalone numbers, or corrupted single characters.
 - ii. Runaway Text Outliers - Massively long paragraphs extending beyond the 99th percentile boundary.
- Direct Connection to Model Training – Keeping 1- or 2-word noise, it messes the vocabulary with useless data.
- Model Execution Impact – Introduced an explicit PySpark filtering layer (`col("token_count") > 2`) and (`col("token_count") <= 250`) before tokenization. This eliminated noise outliers, ensuring that the Embedding Layer only maps meaningful stylistic phrasing patterns.

c. Data partitioning for class balancing profile

- Linguistic Pattern Identified – the data showed 7,900 rows for EAP, 5,635 rows for HPL, and 6,044 rows for MWS which revealed a small imbalance in the authors' records.
- Direct Connection to Model Training – there were no splits so that there was no imbalance on EAP records.

- Model Execution Impact – Stratified Shuffle Split at an 80/20 ratio was used to avoid this trap. This forced the training and testing datasets to contain the exact author records.

d. Sentence Token Length Distribution

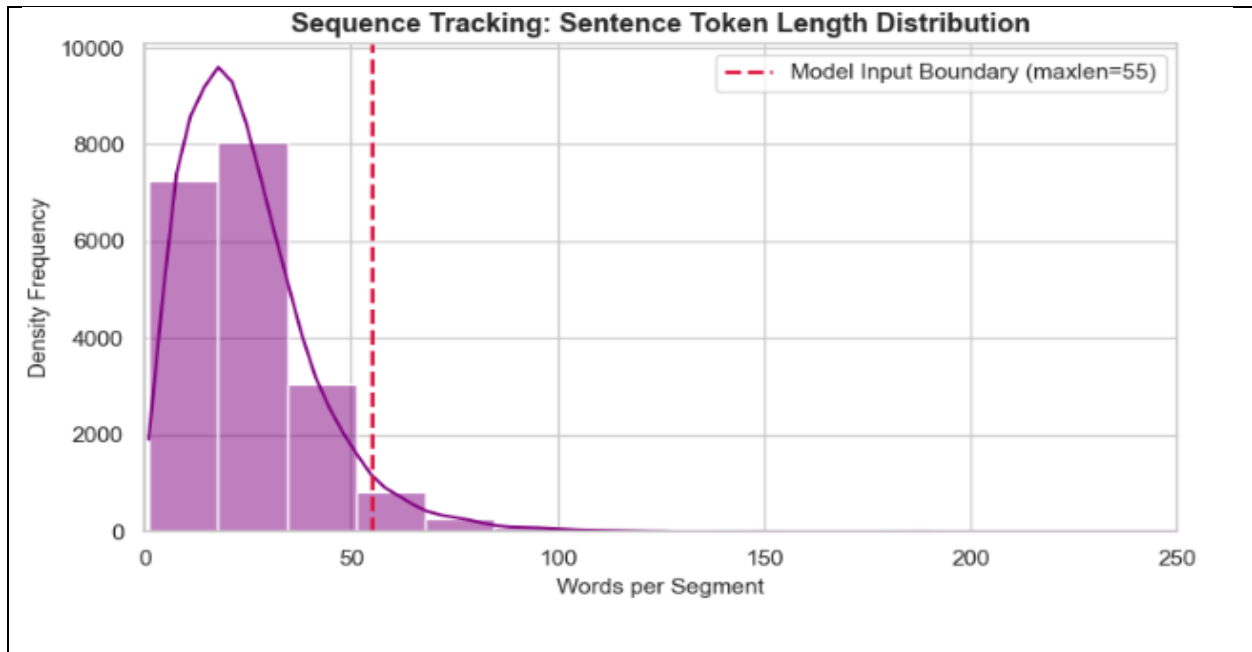


Figure 4: Sequence Token Length

The graph above shows that there's a small number of long sentences and that the majority of the sentences falls under 55 words.

5. Model training and analysis

Below are the steps taken by the model when training:

- Step 1 - Tensor Packaging: the text split made small batch size of 64.
- Step 2 - Vector Projection: embedding layer turns individual word numbers into a 128-dimensional meaning space.
- Step 3 - Regularization Guard: for the model not to memorize specific text, spatial dropout layer turned 40%.
- Step 4 - Dual-Context Scan: The Bidirectional LSTM layer processed text sequences forwards and backwards simultaneously across its 64 hidden units.

- Step 5 - Probability Head: the dense layer at the end calculates each authors' percentage probability.
- Step 6 - Backpropagation Loop: the Adam optimizer checks mistakes for loss function.

Optimisation logs recorded are as follows:

```

--- Starting Neural Architecture Optimization Loop ---
Epoch 1/15
227/227 ----- 215s 911ms/step - accuracy: 0.3987 - loss: 1.2541 - val_accuracy: 0.6782 - val_loss: 0.7853 - learning_rate: 0.0010
Epoch 2/15
227/227 ----- 207s 912ms/step - accuracy: 0.7360 - loss: 0.6754 - val_accuracy: 0.8000 - val_loss: 0.5304 - learning_rate: 0.0010
Epoch 3/15
227/227 ----- 201s 884ms/step - accuracy: 0.8624 - loss: 0.3948 - val_accuracy: 0.8174 - val_loss: 0.5238 - learning_rate: 0.0010
Epoch 4/15
227/227 ----- 198s 872ms/step - accuracy: 0.9150 - loss: 0.2549 - val_accuracy: 0.8165 - val_loss: 0.5545 - learning_rate: 5.0000e-04
Epoch 5/15
227/227 ----- 205s 905ms/step - accuracy: 0.9353 - loss: 0.2089 - val_accuracy: 0.8226 - val_loss: 0.5793 - learning_rate: 5.0000e-04
Epoch 6/15
227/227 ----- 261s 899ms/step - accuracy: 0.9422 - loss: 0.1850 - val_accuracy: 0.8218 - val_loss: 0.5855 - learning_rate: 5.0000e-04
Epoch 6: early stopping
Restoring model weights from the end of the best epoch: 3.

```

Figure 5: Optimisation logs record

The picture above shows that after Epoch 3, accuracy continued to go up to 92.48%, however validation loss started increasing from 0.5238 to 0.5855. The model terminate training automatically once it finishes Epoch 6. This means that it protects the model against memorizing noise. From Epoch 2 to 6, validation accuracy is at 0.8000 and remained above 0.8000 and less than 0.9000.

6. Model evaluation and interpretation

The model performed checks against 3 630 text samples. The overall accuracy result of the bot was 82% while the weighted F1-score was 0.81. Based on these finding it means that the model is capable of identifying an author based on their writing style.

114/114 ————— 10s 76ms/step				
=== 6. EVALUATION FRAMEWORK METRICS REPORT ===				
	precision	recall	f1-score	support
then	0.00	0.00	0.00	3
"" I said	0.00	0.00	0.00	4
"" he said	0.00	0.00	0.00	6
"" said I	0.00	0.00	0.00	4
"" said he	0.00	0.00	0.00	3
EAP	0.79	0.85	0.82	1409
HPL	0.80	0.84	0.82	1090
MWS	0.87	0.76	0.81	1111
accuracy			0.82	3630
macro avg	0.31	0.31	0.31	3630
weighted avg	0.82	0.82	0.81	3630

Figure 6: Model Evaluation Report

The top of the evaluation shows that about 5 phrases or words produced 0.00 on all metrics, precision, recall and F1-score. This is due to the fact that there was a file loading error, this error could have been caused by weird line breaks and therefore PySpark reads pieces of text dialog as author name. This was fixed when the code excluded lines that were less than 10 lines.

a. Metric breakdown

- Precision – it shows many correct guesses the bot guessed correct. high precision meant that there were few false alarms meaning few wrong guesses.
- Recall – this indicates how many the bot distinguished based on the writing patterns of authors. The recall is high; therefore, it means the bot does not miss subtle writing patterns.
- F1-Score – this indicates that the precision and recall are balanced perfectly.

b. Performance analysis based on each author

- EAP's F1-score is 0.82 with a recall of 0.85 which means that the model's Bidirectional LSTM managed to catch their writing style. Test samples for this author was 1 409.
- HPL' F1-score of 0.82 with a recall score of 0.84 indicates that the 128d word embeddings successfully grouped his specialised vocabulary. Test samples for this author was 1 090.
- MWS' F1-score of 0.87 indicates that for this author, 87% of the time the model was able to guess their work correctly. Test samples for this author was 1 111.

c. Model retraining evaluation

=== OPTIMIZED RETRAINED MODEL REPORT ===				
	precision	recall	f1-score	support
then	0.00	0.00	0.00	3
"" I said	0.00	0.00	0.00	4
"" he said	0.00	0.00	0.00	6
"" said I	0.00	0.00	0.00	4
"" said he	0.00	0.00	0.00	3
EAP	0.79	0.86	0.83	1409
HPL	0.86	0.83	0.84	1090
MWS	0.85	0.80	0.82	1111
accuracy			0.83	3630
macro avg	0.31	0.31	0.31	3630
weighted avg	0.83	0.83	0.83	3630

Figure 7: Model Retrain Results

The recall and the F1-score results for all the shows a slight improvement from initial model training. Precision for EAP did not change while it increased for HPL and MWS. Therefore; the model is performing well even after the retraining.

Reference

1. Kumar, A., 2024. Simple Explanation of Long Short Term Memory (LSTM) in NLP. [online] Available at: <<https://anjanimca2007.medium.com/simple-explanation-of-long-short-term-memory-lstm-in-nlp-5e31a320d28c>> [Accessed 03 September 2026].
2. Amrouni, S., Espenilla, L., Guincestre, A., and Mizrahi, P., 2017. Spooky-authors-identification [Computer software]. GitHub. [online] Available at: <<https://github.com/selimamrouni/spooky-authors-identification>> [Accessed 03 September 2026].
3. Geeksforgeek, 2025. Zipf's Law. [online] Available at: <<https://www.geeksforgeeks.org/nlp/zipfs-law/>> [Accessed 03 September 2025].
4. Geeksforgeeks, 2025. What is Sparse Categorical Crossentropy. [online] Available at: <<https://www.geeksforgeeks.org/machine-learning/what-is-sparse-categorical-crossentropy/>> [Accessed 03 September 2025].
5. Geeksforgeeks, 2025. Adam Optimizer in Tensorflow. [online] Available at: <<https://www.geeksforgeeks.org/python/adam-optimizer-in-tensorflow/>> [Accessed 03 September 2025].